

作业2

24371277 刘哲晗

问题1

空闲区按地址排列：10KB、4KB、20KB、18KB、7KB、9KB、12KB、15KB

连续请求：12KB、10KB、9KB

(1) FirstFit（首次适应）

从低地址开始找第一个能满足要求的空闲区：

- 12KB请求：找到20KB（前两个10KB、4KB都不够，第三个是20KB），分配后剩余8KB
- 10KB请求：找到第一个空闲区10KB，正好分配
- 9KB请求：找到9KB空闲区

选中的空闲区：20KB、10KB、9KB

(2) BestFit（最佳适应）

找能满足要求的最小空闲区：

- 12KB请求：最小能满足的是12KB，正好分配
- 10KB请求：最小能满足的是10KB，正好分配
- 9KB请求：最小能满足的是9KB，正好分配

选中的空闲区：12KB、10KB、9KB

(3) WorstFit（最坏适应）

找能满足要求的最大空闲区：

- 12KB请求：最大的空闲区是20KB，分配后剩余8KB
- 10KB请求：剩余最大的是18KB，分配后剩余8KB
- 9KB请求：剩余最大的是15KB，分配后剩余6KB

选中的空闲区：20KB、18KB、15KB

(4) NextFit（下次适应）

从上次查找结束的位置继续查找：

- 12KB请求：从开头找，找到20KB，分配后剩余8KB
- 10KB请求：从20KB之后继续找，找到18KB（不够）→ 找下一个10KB（已回到开头）
- 9KB请求：从10KB之后继续找，找到9KB

选中的空闲区：20KB、10KB、9KB

问题2

逻辑地址、物理地址、地址映射

逻辑地址

也称为虚拟地址，是程序运行时CPU（ALU）产生的地址。它是相对于进程地址空间起始位置的偏移量。

- 每个进程都有自己独立的、连续的逻辑地址空间（如0~4GB）。
- 逻辑地址不直接对应物理内存位置，需要经过地址转换（MMU）才能访问实际内存。

例子：在C语言中打印指针变量的值 `printf("%p", &a)`，输出的就是逻辑地址。

物理地址

是内存单元实际的硬件地址，即物理内存中的真实地址，直接对应内存芯片上的存储单元。

- 物理地址空间是整个计算机系统共享的，所有进程共享同一个物理地址空间。

地址映射（地址转换）

将程序使用的**逻辑地址**转换为**实际的物理内存地址**的过程。现代操作系统由MMU（内存管理单元）硬件自动完成这个转换过程，使用页表或段表作为映射依据。

问题3

页式存储管理中的页表、快表及地址转换过程

为什么要设置页表？

页表是每个进程私有的数据结构，记录了该进程的**逻辑页号（下标）到物理页框号（内容）的映射关系**。由于分页机制中逻辑上相邻的页在物理内存中不一定相邻，页表作为“地址转换数组”，让MMU能够查找到每个逻辑页对应的物理存储位置。没有页表，系统就无法知道某页在内存中的具体位置。

为什么要设置快表（TLB）？

页表存放在物理内存中，纯分页系统访问一个数据需要**2次访存**（1次查页表+1次取数据），二级页表则需要3次访存，这会导致访存效率严重下降。

快表（Translation Lookaside Buffer，翻译后备缓冲器）是CPU内部的高速缓存，专门存放最常用的页表条目。利用**局部性原理**，程序在短时间内会频繁访问相同的页面，将这些映射关系缓存到快表中，TLB命中时可以跳过内存页表查找，极大提高地址转换速度。

页式地址转换过程：

1. **地址拆分：** CPU发出的逻辑地址被自动拆分为**页号P**和**页内偏移W**两部分
2. **TLB查找：** 用页号P查询快表（TLB）
 - 如果TLB命中：直接得到物理页框号，跳过内存查表步骤
 - 如果TLB不命中：继续下一步

3. **页表查找**：MMU通过页表基址寄存器找到进程页表，**越界检查**，如果页号大于等于页表长度，触发地址越界中断。然后根据页号P作为索引，从页表中读出对应的物理页框号，同时将该映射更新到TLB
4. **物理地址合成**：将物理页框号（高位）与页内偏移W（低位）拼接，形成最终的物理地址
5. **访问内存**：使用合成的物理地址访问内存获取数据

问题4

缺页中断的处理流程

当CPU访问的页面不在物理内存中（页表项的驻留位为0）时，MMU会触发缺页中断，进入内核态处理。完整处理流程共9步：

1. **现场保护**：陷入内核态，保存操作系统及当前用户进程的寄存器状态和上下文信息
2. **页面定位**：从硬件寄存器或通过分析指令，确定发生缺页的虚拟页面地址
3. **权限检查**：检查该虚拟地址的有效性及保护位。如果是非法访问（如访问未分配地址、写只读页），则终止该进程（发送段错误信号）
4. **找空闲页框**：在物理内存中查找空闲的页框。如果没有空闲页框，调用页面置换算法选择一个要淘汰的页面
5. **旧页面写回**：如果选中的淘汰页面已被修改过（脏位为1），需要先将其内容写回磁盘，同时将该页框标记为"忙"状态
6. **新页面调入**：启动磁盘I/O操作，将需要的新页面从磁盘（交换分区或可执行文件）复制到分配的物理页框中，此过程CPU可切换执行其他进程
7. **更新页表、TLB**：磁盘I/O完成后产生中断，操作系统更新页表项（设置驻留位为1，填入物理页框号，清除脏位和引用位），标记页框为正常状态
8. **恢复现场**：恢复缺页中断前的进程状态，特别将程序指针指向引发缺页的那条指令
9. **继续执行**：重新执行引发缺页中断的指令，此时该页面已在内存中，可以正常访问

问题5

38位虚拟地址，32位物理地址

(1) 多级页表相比一级页表的主要优点：

- **节省内存空间**：一级页表要求连续存放，对于大地址空间页表本身占用巨大内存（如38位地址、4KB页面时一级页表需要 2^{26} 个PTE，约256MB连续内存）。多级页表可以将页表本身分页存放，**只需要将当前用到的页表部分调入内存**，大幅减少页表占用的内存。
- **无需连续内存**：多级页表的各级页表可以离散存放在不同的物理页框中，不要求大块连续内存，降低了内存分配的压力和碎片问题。
- **灵活性更高**：可以为不同的地址空间区域设置不同的映射权限和属性，更便于实现内存保护和共享。

(2) 二级页表的地址域分配：

已知：

- 页面大小 = 16KB = 2^{14} 字节 → **页内偏移占14位**
- 每个页表项 = 4字节
- 每个页面可容纳的页表项数 = $16KB / 4B = 4096 = 2^{12}$ 个

所以：

- 第二级页表（页表）：每个页表占1页，可映射 2^{12} 个页，因此**第二级页表号需要12位**
- 第一级页表（页目录）：虚拟地址共38位，减去页内偏移14位，再减去第二级页表号12位，剩余12位

最终地址结构：

- 第一级页表号（页目录）：**12位**
- 第二级页表号：**12位**
- 页内偏移：**14位**

验证： $12 + 12 + 14 = 38$ 位

问题6

页面访问模式：周期性循环 + 随机访问，如0,1,2...511,431,0,1,2...511,332...

(1) LRU、FIFO和Clock算法的效果：

LRU（最近最少使用）：

- 效果：相对较好但仍不理想。由于访问是周期性的，当访问到序列末尾的页时，LRU会置换最久未使用的页（序列开头的页），但下一个周期马上又要访问这些刚被置换出去的页，导致频繁缺页。
- 问题：LRU追踪“最近使用”，但这种周期性访问模式中，“**最久未使用**”恰恰是“**马上就要使用**”的，**LRU的预测方向与实际访问模式相反**。

FIFO（先进先出）：

- 效果：最差。FIFO按进入顺序置换，会把最早进入的页（就是循环开头的0、1、2...）置换出去，而这些页在下一轮循环马上就要用到，产生大量**抖动**。
- 问题：存在**Belady现象**风险，且完全不考虑页面访问频率，对周期性模式适应性极差。

Clock（时钟/最近未使用）：

- 效果：比FIFO好，但仍然不够。Clock算法给被访问过的页第二次机会，循环中的大部分页在本轮被访问到时会将访问位为1，得到保护不被置换。
- 问题：当指针转完一整圈时，所有页的访问位都会被清0，下一轮开始时还是会出现大量缺页，周期性特征仍未被有效利用。

(2) 500个页框时，优于LRU的算法设计：

基于工作集的循环检测算法（类Working Set算法）

设计思路：

1. **识别周期性模式**：观察到访问序列核心是0~511的稳定循环（共512页），只是每轮结尾出现一个随机页（431、332等），这些随机页只出现一次就不再使用。
2. **优先保护循环主体**：500个页框接近循环主体的大小，应该尽量保留0~499这些高频循环访问页，让它们常驻内存不被置换。
3. **“牺牲页框”策略**：预留1~2个页框专门用于容纳那些只出现一次的随机页面（如431、332），这些页面用完后立即成为置换首选，绝不占用宝贵的常驻页框。

4. **具体实现**：在页表项中增加"访问频率计数"或"年龄"字段，对于反复出现的页面提高其保护级别；或者采用工作集时钟算法（WSClock），对于年龄大于窗口但仍在循环中的页面给予特殊保护。

相比LRU的优势：LRU会把刚用完、即将进入下一轮循环的页（如0、1、2）置换出去，而本算法识别出这些页很快就会被再次访问，始终保留在内存中。500个页框几乎能装下整个循环主体，缺页率将远低于LRU。

问题7

伙伴系统（Buddy System）的空闲块合并

释放内存时的合并过程：

1. 将释放的内存块加入对应大小的空闲链表
2. 计算该内存块的**伙伴地址**（buddy address）
3. 检查伙伴块是否也处于空闲状态：
 - 如果伙伴也空闲：将两个伙伴块合并，形成一个大小为原来2倍的新块，从原大小链表中移除这两个块，将合并后的新块加入上一级大小的空闲链表
 - 如果伙伴不空闲：合并结束，当前块留在原链表
4. 对合并后的新块，**重复步骤2-3**，继续尝试与它的伙伴合并，直到无法继续合并为止（直到伙伴已被分配，或已达到系统最大内存块）

两个内存块成为"伙伴"必须满足的三个条件：

1. **大小相等**：两个块的大小必须完全相同，都是 2^k 字节
2. **地址连续且对齐**：它们在物理地址空间中是连续的，且第一个块的起始地址是 $2^{(k+1)}$ 字节对齐的（即两个块组成了一个自然的 $2^{(k+1)}$ 字节块）
3. **互斥性**：对于任意内存块，有且仅有一个伙伴块

伙伴地址计算公式：假设块大小为 2^k ，块的起始地址为A，则其伙伴地址为：

如果 $A \bmod 2^{(k+1)} == 0 \rightarrow$ 伙伴地址 = $A + 2^k$ （当前块是前半部分）
否则 $\rightarrow$ 伙伴地址 = $A - 2^k$ （当前块是后半部分）

问题8

二级页表，物理访存100ns，TLB访问10ns，TLB命中率95%，Cache访问5ns，Cache命中率98%

(1) 最佳情况（TLB命中且Cache命中）下的访问时间：

最佳情况是最快路径：

1. TLB访问：10ns（命中，无需访存查页表）
2. Cache访问：5ns（命中，无需访问物理内存）

总时间 = 10ns + 5ns = 15ns

(2) 平均内存访问时间（EAT）计算：

需要考虑所有组合情况的概率并加权求和：

第一阶段：地址翻译（TLB + 页表查询）

- TLB命中（95%）：耗时 = 10ns（TLB访问）
- TLB未命中（5%）：需要访问二级页表 → 2次内存访问
 - 页表项访问也可能被Cache缓存，需考虑Cache命中率
 - 平均耗时 = $10\text{ns} + [98\% \times 5\text{ns} + 2\% \times 100\text{ns}] \times 2$

第二阶段：数据访问

- 无论TLB是否命中，最终都要用得到的物理地址访问数据
- 数据访问平均耗时 = $98\% \times 5\text{ns} + 2\% \times 100\text{ns}$

完整计算：

地址翻译平均耗时：= $95\% \times 10\text{ns} + 5\% \times [10\text{ns} + 2 \times (98\% \times 5\text{ns} + 2\% \times 100\text{ns})] = 9.5\text{ns} + 5\% \times [10\text{ns} + 2 \times (4.9\text{ns} + 2\text{ns})] = 9.5\text{ns} + 5\% \times [10\text{ns} + 13.8\text{ns}] = 9.5\text{ns} + 5\% \times 23.8\text{ns} = 9.5\text{ns} + 1.19\text{ns} = \mathbf{10.69\text{ns}}$

数据访问平均耗时：= $98\% \times 5\text{ns} + 2\% \times 100\text{ns} = 4.9\text{ns} + 2\text{ns} = \mathbf{6.9\text{ns}}$

平均内存访问时间EAT = 地址翻译耗时 + 数据访问耗时 = $10.69\text{ns} + 6.9\text{ns} = \mathbf{17.59\text{ns}}$